An Energy Efficient SDN Controller Placement with Delay Constraints

Tomofumi Kondo*, Luis Guillen†, Satoru Izumi‡, Toru Abe*†, Takaaki Mizuki*† and Takuo Suganuma*†
*Graduate School of Information Sciences, Tohoku University, Japan
†Cyberscience Center, Tohoku University, Japan
‡National Institute of Technology, Sendai College, Japan
Email: tomokon@dc.tohoku.ac.jp, lguillen@tohoku.ac.jp, izumi@sendai-nct.ac.jp, {mizuki, beto, suganuma}@tohoku.ac.jp

***Abstract*—Software-Defined Networking (SDN) separates the control from the data plane in the network infrastructure to flexibly and efficiently manage the resources. Since the controller is the core of the control plane, multiple instances need to be placed within the network to increase performance, reliability, and fault tolerance, among others. However, deciding the number and where to place the allotted controllers is not trivial; it is known as the Controller Placement Problem (CPP). This paper considers the energy-efficiency aspect of the network infrastructure to solve the CPP. To do so, we designed a model that includes delay constraints, whose effectivity was assessed and confirmed to save about 20-30% more energy than a model that only considers the number of controllers. Moreover, it is confirmed that execution time can be decreased by 100 times while keeping the energy consumption low by reducing the number of pre-calculated paths between the SDN switches and the controllers.**

***Index Terms*—SDN, Controller Placement Problem, Energy Efficiency, BIP**

I. Introduction

Software Defined Networking (SDN) is emerging as a new network architecture. SDN separates the infrastructure into two planes for effective control. The control plane has a controller that collects information about the network and modifies the flow rules of network devices which are part of the data plane in charge of forwarding incoming flows according to the rules set by the controller. These devices are called SDN switches.

SDN brings flexibility to a network because its freely programmable control plane centrally manages the entire network. Furthermore, SDN can manage a network efficiently because controllers collect various information from the entire network and can understand the state of the network in detail.

However, an SDN infrastructure with a single controller cannot cope with the basic requirements in terms of performance, fault tolerance, or others. For instance, the delays between SDN switches and controllers become too high in a large-scale network, or the entire network becomes inoperable if the only controller in the network fails. Therefore, SDN needs multiple controllers to overcome those issues. However, in this scenario (i.e., multi-controller SDN infrastructure), we need to determine the appropriate number of controllers, their locations, and which controller should manage each switch. Determining the optimal values of the aforementioned parameters is known as the Controller Placement Problem (CPP) [1]. There are various possible purposes when solving CPP. For example, network performance, fault tolerance, reliability, resiliency, load balancing across controllers and switches, cost, and energy efficiency.

In this work, we focus on energy efficiency and network performance to solve the CPP. To do so, this research uses Binary Integer Programming (BIP) to solve the CPP considering these constraints. Initially, we present the system model, then evaluate its performance in a practical context. From the preliminary result, we confirmed that the model can derive the controller's placements that satisfy network performance constraints while minimizing power consumption in the network. In addition, we also confirmed the relation between some parameters and the execution time.

II. Related Work

Although the CPP problem has been vastly investigated over the past decade, to the best of our knowledge, few of them address energy and performance constraints. For instance, Greco [2] is among the first solutions that propose an energy-efficient CPP model considering power consumption on links and flow delays between SDN switches and controllers. Greco saves power by deactivating unused links in the network. However, only the number of active links is taken into account as a metric related to energy efficiency. Hence, the power consumption on controllers and switches and the power consumption that increases with the load on network devices are not considered in this work.

Another inspiring work is EnPlace [3] which proposes an energy-efficient CPP model that reduces power consumption on links and controllers considering flow delays between SDN switches and controllers on a dynamic IoT network. EnPlace considers two parts of power consumption. The first is base power consumption that occurs in a certain amount if a network device is active. The other is power consumption that increases dynamically with the load. However, EnPlace is incomplete because it minimizes the sum of the power-usage rates on each network device, not the absolute amount of power consumption that occurs in the entire network. In addition, EnPlace does not describe mathematically how to select paths between controllers and SDN switches, and how link activation is determined.

On the other hand, LiDy+ [4] proposes a CPP model with dynamic flows and the heuristic algorithm whose computa-

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:16:45 UTC from IEEE Xplore. Restrictions apply.

tional complexity is small enough to be used with large-scale networks. LiDy+ considers the number of controllers, the load of SDN switches and controllers, the power consumption on controllers, and the maintenance cost. However, the energy consumed by links and switches is not part of their model.

Balakiruthiga & Deepalakshmi [5] propose a CPP model and dynamic routing considering the power consumption of links and flow delays between controllers and SDN switches. However, this work does not consider the power consumption of controllers and switches.

Finally, Hu et al. [6] proposes a heuristic called IGCPA which designs a CPP model with dynamic flows to maximize the energy-saving ratio by reducing active links. However, IGCPA does not consider the power consumption on controllers and switches and the power consumption that increases with the load on network devices.

III. System Model

To fill the void in the existing work, we propose a model to solve the CPP with energy-efficient and delay constraints as described in this section. The proposed model considers the power consumption on SDN switches in addition to the one on controllers and links. Furthermore, the proposed model also considers the power consumption based on the load of links, SDN switches, and controllers for a more accurate power consumption model.

Table I summarizes the notation used in this model. Moreover, we detail each of the elements in the following subsections.

A. Network

We consider a network as an undirected single graph $G = (V, E)$. V is a group of nodes in the network and E is a group of links. $S(= V)$ is a group of SDN switches. The controllers will be placed at several locations in V collocated with SDN switches. Let the controller located on node $i \in V$ be c_j. Similarly, let C be a group of controllers, and let $e_{mn} \in E$ be a weighted link between node m and n. A sample network topology is shown in Fig. 1.

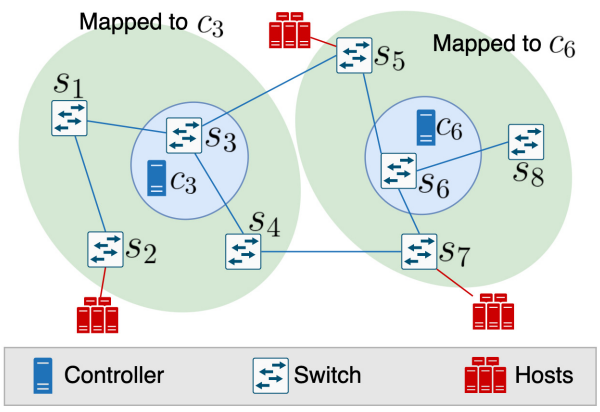

Fig. 1. Network Model

TABLE I
Notation in the Proposed Model

Symbol	*Definition*
$G(V, E)$	Network Graph
V	Set of nodes in the network graph
E	Set of links in the network graph
S	Set of SDN switches
C	Set of controllers
L	Number of pre-calculated paths
P_{ij}	Set of paths between s_i and c_j
P^E_{mn}	Set of paths through e_{mn}
D	Ratio of SDN switches connected to hosts
F^S_i	Number of flows toward s_i
F^C_j	Number of flows toward c_j
F^E_{mn}	Number of flows through e_{mn}
Cap^S	Capacity of an SDN switch
Cap^C	Capacity of a controller
Cap^E	Capacity of a link
W_{ij}	Propagation delay between s_i and c_j
W^E_{mn}	Propagation delay of e_{mn}
W_{th}	Delay threshold
PC^E_{mn}	Power consumption on e_{mn}
PC^{E0}	Base power consumption on a link
PC^{Ef}	Power consumption coefficient on a link
PC^S_i	Power consumption on s_i
PC^{S0}	Base power consumption on an SDN switch
PC^{Sf}	Power consumption coefficient on an SDN switch
PC^C_j	Power consumption on c_j
PC^{C0}	Base power consumption on a controller
PC^{Cf}	Power consumption coefficient on a controller

Next, we define some binary variables. First, the purpose of CPP is to determine how many and where to place controllers. So we define the controller placement variable.

$$c(j) = \begin{cases} 1 & \text{if a controller is placed to the node } j \in V \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

Second, we assume that each SDN switch is managed by only one controller to update flow rules programmatically. The variable for mapping between SDN switches and controllers is described below.

$$sc(i, j) = \begin{cases} 1 & \text{if the SDN switch } s_i \in S \text{ is managed by} \\ & \text{the controller } c_j \in C \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

Next, unused SDN switches and links are deactivated to save power. So we define the activation variable for SDN switches and links.

$$s(i) = \begin{cases} 1 & \text{if the SDN switch } s_i \in S \text{ is activated} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

$$e(m, n) = \begin{cases} 1 & \text{if the link } e_{mn} \in E \text{ is activated} \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:16:45 UTC from IEEE Xplore. Restrictions apply.

B. Path

We should select control paths between SDN switches and controllers from pre-calculated paths between nodes. P_{ij} is a group of paths between the SDN switch $s_i \in S$ and the controller $c_j \in C$. The maximum number of calculated paths between each node is denoted by L. $p_{ij}^l \in P_{ij}$ is the l-th shortest path between the SDN switch $s_i \in S$ and the controller $c_j \in C$. p_{ij}^l consists of nodes through which the path passes. P_{mn}^E is a group of paths through the link $e_{mn} \in E$.

We should select only one path between each mapped SDN switch and controller from a path group, so we define the variable for path selection.

$$p(i,j,l) = \begin{cases} 1 & \text{if the path } p_{ij}^l \text{ is used} \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

C. Flow

In the case of the flows, we defined D as a ratio of SDN switches directly connected to hosts. Hosts send packets to a connected SDN switch, and that SDN switch forwards them to the proper destination. When an SDN switch on a forwarding path receives an unknown packet, it sends a PacketIn request to its mapped controller. The number of flows the SDN switch connected to the hosts receives is expressed as F_i^S.

The number of flows the controller $c_j \in C$ receives is the sum of the PacketIn flows sent by SDN switches mapped to that controller. It is described as $F_j^C = \sum_{s_i \in S} sc(i,j)\ F_i^S$. The number of flows through the link $e_{mn} \in E$ is the sum of flows on the way to and from through paths including that link. Thus, it is described as $F_{mn}^E = 2 \sum_{p_{ij}^l \in P_{mn}^e} p(i,j,l)\ F_i^S$.

Each type of network device has its capacity. The number of flows through each piece of a network device should not exceed its capacity. Capacities for SDN switches, controllers, and links are denoted as Cap^S, Cap^C, and Cap^E respectively.

D. Delay

There will be a delay while a control packet flows between an SDN switch and its controller, which can be calculated from a used path and a delay of links on that path. It is described as $W_{ij} = \sum_{p_{ij}^l \in P_{ij}} p(i,j,l) \sum_{e_{mn} \in p_{ij}^l} W_{mn}^E$. W_{mn}^E is a delay of the link $e_{mn} \in E$.

The delay threshold is denoted as W_{th}. None of the delays between an SDN switch and a controller exceeds that threshold.

E. Power Consumption

We aim to save power in the network, so we created a power consumption model for each type of network device.

First, the power consumption model for the link $e_{mn} \in E$ is described as $PC_{mn}^E = e(m,n)PC^{E0} + F_{mn}^E PC^{Ef}$. PC^{E0} is the base power consumption on a link. We assume that the amount of power is consumed at a constant if one link is activated. PC^{Ef} is the power consumption coefficient on a link. That amount of power multiplied by the number of flows through that link is consumed at each link.

Second, the power consumption model for the SDN switch S_i is described as $PC_i^S = s(i)PC^{S0} + F_i^S PC^{Sf}$. PC^{S0} and PC^{Sf} are the base power consumption and the power consumption coefficient on an SDN switch respectively.

Finally, the power consumption model for the controller c_j is described as $PC_j^C = c(j)PC^{C0} + F_j^C PC^{Cf}$. PC^{C0} and PC^{Cf} are the base power consumption and power consumption coefficient on a controller respectively.

IV. Problem Formulation

We modeled CPP as a BIP to determine the appropriate number of controllers and their locations such that power is saved and the condition for the allowable delay is satisfied.

A. Objective Function

We defined an objective function to minimize total power consumption in the network.

$$\text{Minimize}_{c,sc,p,e,s} \quad \sum_{e_{mn} \in E} PC_{mn}^E + \sum_{s_i \in S} PC_i^S + \sum_{c_j \in C} PC_j^C \tag{6}$$

Each of these terms expresses the total power consumption of links, SDN switches, and controllers respectively.

B. Constraints

Based on the above objective model, we apply the following constraints for CPP to place controllers appropriately.

$$\sum_{c_j \in C} sc(i,j) = s(i) \quad \forall s_i \in S \tag{7}$$

$$sc(i,j) \le c(j) \quad \forall s_i \in S, c_j \in C \tag{8}$$

$$s(i) = 1 \quad \text{if } F_i^S > 0 \quad \forall s_i \in S \tag{9}$$

$$s(i) \ge e(m,n) \quad \forall e_{mn} \in E \quad \forall s_i \in S \tag{10}$$

$$\sum_{p_{ij}^l \in P_{ij}} p(i,j,l) = sc(i,j) \quad \forall s_i \in S, c_j \in C \tag{11}$$

$$e(m,n) \ge p(i,j,l) \quad \forall p_{ij}^l \in P_{mn}^e \quad \forall e_{mn} \in E \tag{12}$$

$$W_{ij} \le W^{th} \quad \forall s_i \in S, c_j \in C \tag{13}$$

$$F_{mn}^E \le Cap^E \quad \forall e_{mn} \in E \tag{14}$$

$$F_i^S \le Cap^S \quad \forall s_i \in S \tag{15}$$

$$F_j^C \le Cap^C \quad \forall c_j \in C \tag{16}$$

Equation (7) states that each active SDN switch is mapped to only one controller. Next equation (8) states that SDN switches can be mapped to a location where a controller is placed. Then equation (9) restricts that SDN switches that receive flows from hosts should be active. Then equation (10) ensures that SDN switches having at least one link should be active. Equation (11) restricts that only one path should be selected between each mapped SDN switch and its controller.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:16:45 UTC from IEEE Xplore. Restrictions apply.

Equation (12) ensures that links on a used path should be active. Equation (13) states that all delays between mapped SDN switches and controllers should not exceed the delay threshold. Finally, equation (14), (15), and (16) state that the number of flows through each link, SDN switch, and controller should not exceed their capacities respectively.

V. Evaluation

We conducted experiments to evaluate the proposed model. The environment, the key metrics, the schemes, and the results of the experiments are shown in the next sections.

A. Environment

We performed a BIP to determine the controller placement using PuLP 2.7.0 [7] with Python3.10.11 to describe the proposed model and Gurobi Optimizer [8] as a solver. The Evaluation was conducted on VM with Ubuntu 22.04.02 LTS, 8 cores Intel Xeon CPU E5-2650 v4 @ 2.20 GHz, and 8 GB memory.

We used AT&T Core Network Topology with 24 nodes and 57 links as described in The Internet Topology Zoo [9] where to place controllers. The parameters we used in the evaluation are described in Table II

TABLE II
Evaluation Parameters

Parameter	***Value***
Topology	Att (24 nodes and 57 links) [9]
Number of SDN switches	24
Number of controllers	Unlimited
Number of pre-calculated paths (L)	3 (in scheme 1)
The ratio of SDN switches connected to hosts (D)	0.5
Number of flows toward an SDN switch (F_i^S)	between 50K and 100K flows/s
Capacity of an SDN switch (Cap^S)	1000K flows/s
Capacity of a Controller (Cap^C)	2000K flows/s
Capacity of a link (Cap^E)	2000K flows/s
Propagation delay of e_{mn} (W_{mn}^E)	5 µs/km
Delay threshold (W^{th})	10 ms (in scheme 2)
Base power consumption on a link (PC^{E0})	30 Watt [3]
Power consumption coefficient on a link (PC^{Ef})	0.005 Watt/flow [3]
Base power consumption on an SDN switch (PC^{S0})	50 Watt [10]
Power consumption coefficient on an SDN switch (PC^{Sf})	0.002 Watt/flow [10]
Base power consumption on a controller (PC^{C0})	100 Watt [11]
Power consumption coefficient on a controller (PC^{Cf})	0.1 Watt/flow [11]

B. Metrics

We evaluated the following metrics:

- Power consumption: The total amount of power consumption in the network, which is the core of this paper.
- Average delay: Average of all delays between SDN switches and controllers. The higher the average delay, the worse the network performance.
- The worst delay: Highest delay between SDN switches and controllers. This is expected not to exceed the delay threshold.
- Execution time: Time to finish the CPP calculation. The shorter this time, the better.

C. Schemes

We conducted two variants of experiments.

1) Delay threshold: Measure how metrics transition when the delay threshold is gradually increased which means the delay is relaxed to evaluate the effect of the delay threshold on energy efficiency and network performance.
2) The number of paths: Measure how metrics transition when the number of pre-calculated paths changes to evaluate how much execution time can be saved and how energy efficiency and network performance are sacrificed in that case.

In scheme 1, we evaluated the proposed model against another one that only minimizes the number of controllers and does not mind the power consumption, we call this model *min_controllers*. The objective function of the *min_controllers* model is described as Minimize$_{c,sc,p,e,s}$ $\sum_{c_j \in C} c_j$. So, it does not consider the link or switch activation. The other parts of *min_controllers* are the same as our proposed model.

D. Result

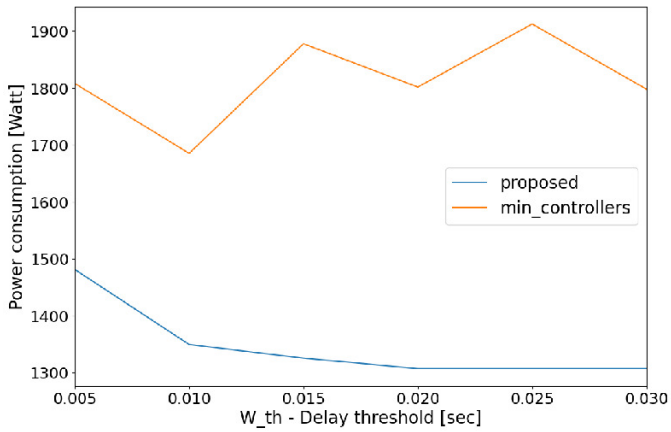

Fig. 2. Power consumption relative to delay threshold

1) Delay Threshold: Fig. 2 shows the transition of the power consumption relative to the delay threshold (W_{th}). From this, it is confirmed that the higher W_{th}, the lower the power consumption up to 11%. This is because the delay threshold is relaxed, and the controllers can be placed and mapped to SDN switches more freely. Moreover, the proposed model saves between 17 and 32 % of power compared to *min_controllers*. Next, Fig. 3 shows the transition of the average delay relative to W_{th}. As observed, the average delay is lower in our proposed model than in the *min_controllers* model. However, it is worth noting that the proposed model

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:16:45 UTC from IEEE Xplore. Restrictions apply.

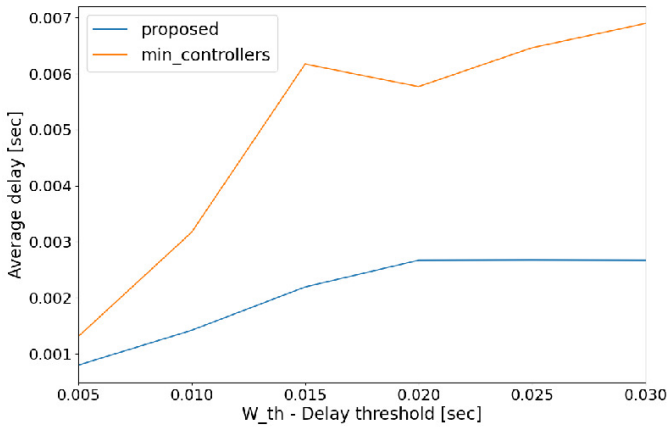

Fig. 3. Average delay relative to delay threshold

uses more controllers than *min_controllers*, as we do not limit this variable. So the distances between controllers and SDN switches are short in our model. The transition of the worst

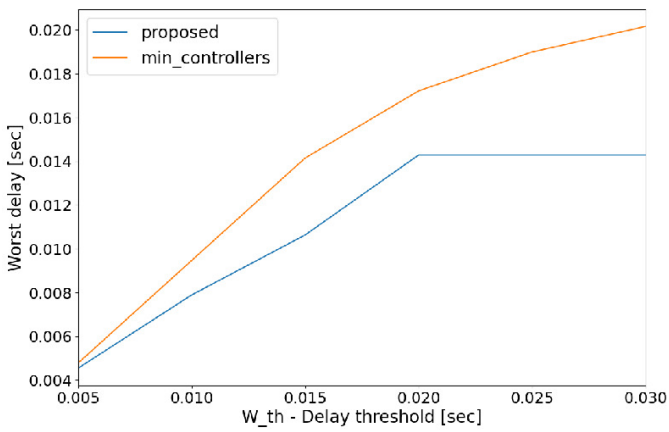

Fig. 4. Worst delay relative to delay threshold

delay relative to W_{th} is shown in Fig. 4. As observed, the worst delay is lower than *min_controllers* at any W_{th}, so the delay constraint is satisfied Finally, the transition of the execution time relative to W_{th} is shown in Fig. 5. What we confirmed in this experiment is that, as the delay threshold is higher, the more time it takes to calculate the placement. The reason for it is that there are more selectable paths when W_{th} become loose, hence the solver struggles to find more energy-efficient controller placement and mappings. Note that the proposed model takes more time than *min_controllers* regardless of the threshold, as the proposed model has more considerations and its computational complexity is relatively high.

2) The Number of Paths: In the next set of experiments, we used the transition of the power consumption relative to the number of pre-calculated paths (L). First, Fig. 6 shows that the power consumption decreases just a little (5 to 6 Watts)

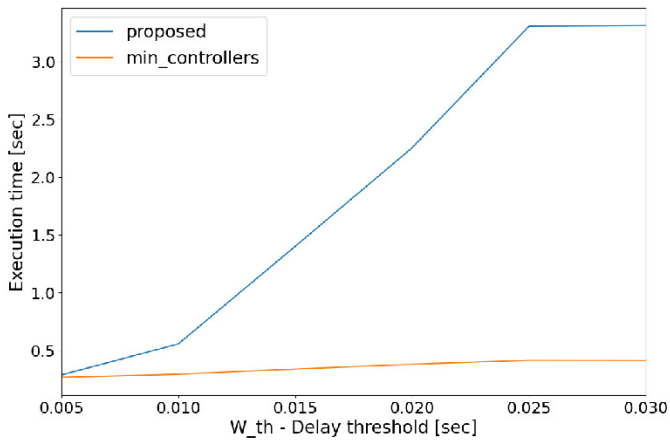

Fig. 5. Execution time relative to delay threshold

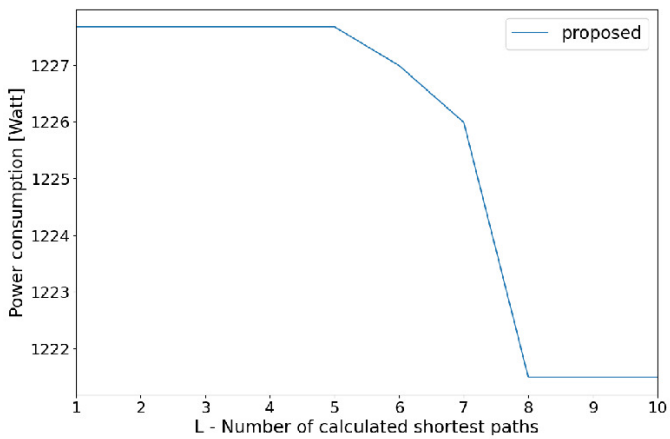

Fig. 6. Power consumption relative to the number of paths

in stages when the number of pre-calculated paths increases. This is because the model can derive more energy-efficient placements and mappings when it has more selectable paths. However, the change in power consumption is small enough to assume that almost only the shortest paths are used in the condition of this time.

Fig. 7 shows how the average delay is influenced by the number of pre-calculated paths. As observed, the average delays become a little lower (up to 0.3 to 0.4 [ms]) if L is large. The reason is that if there are more selectable paths, the used paths are selected to reduce the total number of links in the entire network to save the power on links. Thus, the number of links used in the selected paths decreases so does the average delay. Next, Fig. 8 shows the transition of the worst delay relative to the W_{th}. As observed, in comparison with the average delay, the worst delay increases a little (up to 0.4 to 0.5 [ms]) when L becomes large. This is because longer paths can be used to save the overall power (in case several paths are available). That means that the overall

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:16:45 UTC from IEEE Xplore. Restrictions apply.

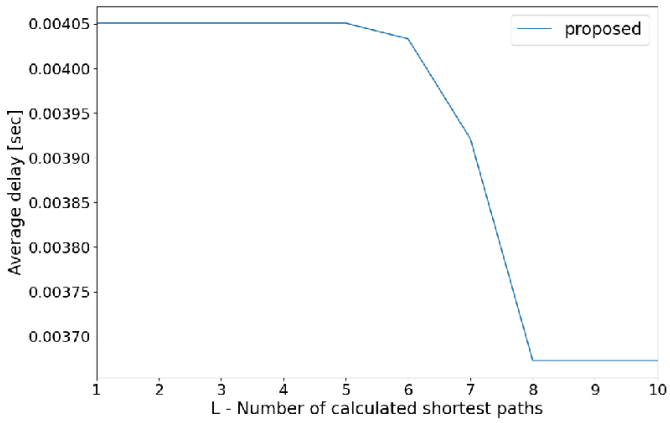

Fig. 7. Average delay relative to the number of paths

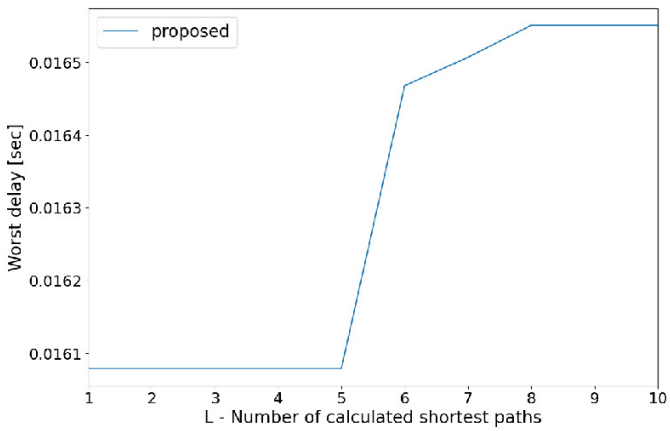

Fig. 8. Worst delay relative to the number of paths

energy efficiency and network performance are improved by sacrificing the performance between the specific switch and controller. Finally, Fig. 9 shows the execution time relative to the W_{th}. As observed, the execution time to solve the CPP increases linearly when the number of pre-calculated paths increases. Therefore, given these results, we can execute CPP in a short time without sacrificing the flexibility to save power consumption by reducing the number of pre-calculated paths.

VI. Conclusion

This paper proposes a model to solve the CPP as BIP to derive the controllers' placement and mappings to SDN switches such that power consumption is minimized and delay constraints are satisfied. From the preliminary results of the evaluation, it is confirmed that our model saves power by 17 to 32 percent compared to a model that only considers the number of controllers. Moreover, we can also conclude that the execution time could be reduced by decreasing the number of pre-calculated paths without hindering power consumption.

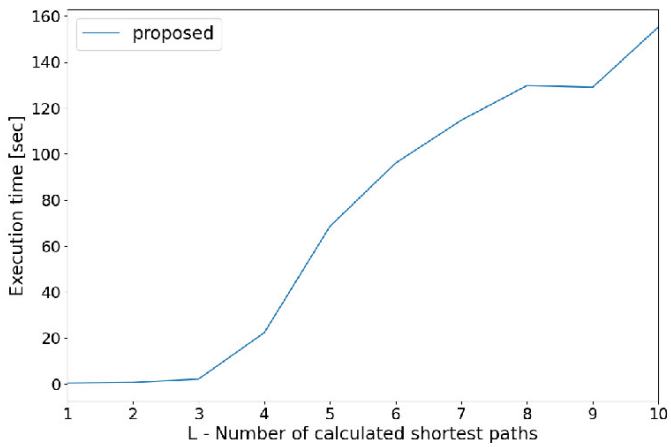

Fig. 9. Execution time relative to the number of paths

As a future work, we plan to improve the power consumption model to be more realistic, consider user-plane flows because we only consider control-plane flows in this work, and propose a heuristic algorithm to solve CPP taking a short time even in a large-scale network. In addition, we plan to evaluate our model in a real SDN environment measuring actual power consumption.

References

[1] B. Heller, R. Sherwood, and N. McKeown, "The controller placement problem," in *Proceedings of the first workshop on Hot topics in software defined networks(HotSDN)*, New York, NY, Aug. 2012, pp. 7–12.

[2] A. Ruiz-Rivera, K.-W. Chin, and S. Soh, "Greco: An energy aware controller association algorithm for software defined networks," *IEEE Commun. Lett.*, vol. 19, pp. 541–544, Jan. 2015.

[3] I. Maity, R. Dhiman, and S. Misra, "Enplace: Energy-aware network partitioning for controller placement in sdn," *IEEE Trans. Green Commun. Netw.*, vol. 7, pp. 183–193, May 2022.

[4] M. T. I. ul Huque, W. Si, G. Jourjon, and V. Gramoli, "Large-scale dynamic controller placement," *IEEE Trans. Netw. Service Manag.*, vol. 14, pp. 63–76, Jan. 2017.

[5] B. Balakiruthiga and P. Deepalakshmi, "A distributed energy aware controller placement model for software-defined data centre network," *Iranian Journal of Science and Technology, Transactions of Electrical Engineering*, vol. 45, pp. 1083–1101, Apr. 2021.

[6] Y. Hu, T. Luo, N. C. Beaulieu, and C. Deng, "The energy-aware controller placement problem in software defined networks," *IEEE Commun. Lett.*, vol. 21, pp. 741–744, Dec. 2016.

[7] I. Dunning, S. Mitchell, and M. O'Sullivan, "Pulp: A linear programming toolkit for python," 2011. [Online]. Available: https://optimization-online.org/?p=11731

[8] L. Gurobi Optimization, "Gurobi optimizer reference manual," 2023. [Online]. Available: https://www.gurobi.com

[9] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The internet topology zoo," *IEEE J. Sel. Areas Commun.*, vol. 29, pp. 1765–1775, Oct. 2011.

[10] F. Kaup, S. Melnikowitsch, and D. Hausheer, "Measuring and modeling the power consumption of openflow switches," in *10th International Conference on Network and Service Management (CNSM) and Workshop*, Nov. 2014, pp. 17–21.

[11] X. Zhang, J. Lu, and X. Qin, "Bfepm: Best fit energy prediction modeling based on cpu utilization," in *2013 IEEE Eighth International Conference on Networking, Architecture and Storage (NAS)*, Jul. 2013, pp. 17–19.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:16:45 UTC from IEEE Xplore. Restrictions apply.